INTERPRETABILITY IN THE WILD: A CIRCUIT FOR INDIRECT OBJECT IDENTIFICATION IN GPT-2 SMALL

Kevin Wang[1], **Alexandre Variengien**[1], **Arthur Conmy**[1], **Buck Shlegeris**[1] **& Jacob Steinhardt**[1,2]
[1]Redwood Research
[2]UC Berkeley
kevin@rdwrs.com, alexandre@rdwrs.com,
arthur@rdwrs.com, buck@rdwrs.com, jsteinhardt@berkeley.edu

arXiv:2211.00593v1 [cs.LG] 1 Nov 2022

ABSTRACT

Research in mechanistic interpretability seeks to explain behaviors of machine learning (ML) models in terms of their internal components. However, most previous work either focuses on simple behaviors in small models or describes complicated behaviors in larger models with broad strokes. In this work, we bridge this gap by presenting an explanation for how GPT-2 small performs a natural language task called indirect object identification (IOI). Our explanation encompasses 26 attention heads grouped into 7 main classes, which we discovered using a combination of interpretability approaches relying on causal interventions. To our knowledge, this investigation is the largest end-to-end attempt at reverse-engineering a natural behavior "in the wild" in a language model. We evaluate the reliability of our explanation using three quantitative criteria–*faithfulness, completeness* and *minimality*. Though these criteria support our explanation, they also point to remaining gaps in our understanding. Our work provides evidence that a mechanistic understanding of large ML models is feasible, pointing toward opportunities to scale our understanding to both larger models and more complex tasks. Code for all experiments is available at https://github.com/redwoodresearch/Easy-Transformer.

1 INTRODUCTION

Transformer-based language models (Vaswani et al., 2017; Brown et al., 2020) have demonstrated an impressive suite of capabilities but largely remain black boxes. Understanding these models is difficult because they employ complex non-linear interactions in densely-connected layers and operate in a high-dimensional space. Despite this, they are already deployed in high-impact settings (Zhang et al., 2022; Caldarini et al., 2022), underscoring the urgency of understanding and anticipating possible model behaviors. Some researchers have argued that interpretability is critical for the safe deployment of advanced machine learning systems (Hendrycks & Mazeika, 2022).

Work in mechanistic interpretability aims to discover, understand, and verify the algorithms that model weights implement by reverse engineering model computation into human-understandable components (Olah, 2022; Meng et al., 2022; Geiger et al., 2021; Geva et al., 2020). By understanding underlying mechanisms, we can better predict out-of-distribution behavior (Mu & Andreas, 2020), identify and fix model errors (Hernandez et al., 2021; Vig et al., 2020), and understand emergent behavior (Nanda & Lieberum, 2022; Barak et al., 2022; Wei et al., 2022).

In this work, we aim to mechanistically understand how GPT-2 small (Radford et al., 2019) implements a simple natural language task. We do so by using *circuit analysis* (Räuker et al., 2022), identifying an induced subgraph of the model's computational graph that is human-understandable and responsible for completing the task.

To discover the circuit, we introduce a systematic approach that iteratively traces important components back from the logits, using a causal intervention that we call "path patching". We supplement this mainline approach with projections in the embedding space, attention pattern analysis, and activation patching to understand the behavior of each component.

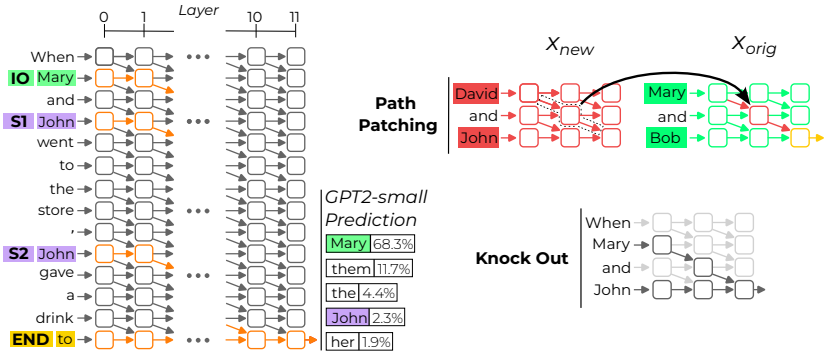

Figure 1: Left: We isolated a *circuit* (in orange) responsible for the flow of information connecting the indirect object "Mary" to the next token prediction. The nodes are attention layers and the edges represent the interactions between these layers. Right: We discovered and validated this circuit using causal interventions, including both path patching and knockouts of attention heads.

We focus on understanding a specific natural language task that we call indirect object identification (IOI). In IOI, sentences such as "When Mary and John went to the store, John gave a drink to" should be completed with "Mary". We chose this task because it is linguistically meaningful and admits an interpretable algorithm: of the two names in the sentence, predict the name that isn't the subject of the last clause.

We discover a circuit of 26 attention heads–1.1% of the total number of (head, token position) pairs–that completes the bulk of this task. The circuit uses 7 categories of heads (see Figure 2) to implement the algorithm. Together, these heads route information between different name tokens, to the end position, and finally to the output. Our work provides, to the best of our knowledge, the most detailed attempt at reverse-engineering an end-to-end behavior in a transformer-based language model.

By zooming in on a crisp task in a particular model, we obtained several insights about the challenges of mechanistic interpretability. In particular:

- We identified several instances of heads implementing redundant behavior (Michel et al., 2019). The most surprising were "Backup Name-Mover Heads", which copy names to the correct position in the output, but only when regular Name-Mover Heads are ablated. This complicates the search for complete mechanisms, as different model structure is found when some components are ablated.
- We found known structures (specifically induction heads (Elhage et al., 2021)) that were used in unexpected ways. Thus mainline functionality of a component does not always give a full picture.
- Finally, we identified heads reliably writing in the *opposite* direction of the correct answer.

Explanations for model behavior can easily be misleading or non-rigorous (Jain & Wallace, 2019; Bolukbasi et al., 2021). To remedy this problem, we formulate three criteria to help validate our circuit explanations. These criteria are **faithfulness** (the circuit can perform the task as well as the whole model), **completeness** (the circuit contains all the nodes used to perform the task), and **minimality** (the circuit doesn't contain nodes irrelevant to the task). Our circuit shows significant improvements compared to a naïve (but faithful) circuit, but fails to pass the most challenging tests.

In summary, our main contributions are: (1) We identify a large circuit in GPT-2 small that performs indirect-object identification (Figure 2 and Section 3); (2) Through example, we identify useful techniques for understanding models, as well as surprising pitfalls; (3) We present criteria that ensure structural correspondence between the circuit and the model, and check experimentally whether our circuit meets this standard (Section 4).

2 Background

In this section, we introduce the IOI task, review the transformer architecture, define circuits more formally, and describe a technique for "knocking out" nodes in a model.

Task description. A sentence containing indirect object identification (IOI) has an initial dependent clause, e.g "When Mary and John went to the store", and a main clause, e.g "John gave a bottle of milk to Mary". The initial clause introduces the indirect object (IO) "Mary" and the subject (S) "John". The main clause refers to the subject a second time, and in all our examples of IOI, the subject gives an object to the IO. The IOI task is to predict the final token in the sentence to be the IO. We use 'S1' and 'S2' to refer to the first and second occurrences of the subject, when we want to specify position. We create dataset samples for IOI using 15 templates (see Appendix E) with random single-token names, places and items. We use the notation $\mathrm{p}_{\mathrm{IOI}}$ for the distribution over sentences generated by this procedure.

To quantify GPT-2 small's performance on the IOI task, we use two different metrics: logit difference and IO probability. *Logit difference* measures the difference in logit value between the two names, where a positive score means the correct name (IO) has higher probability. *IO probability* measures the absolute probability of the IO token under the model's predictions. Both metrics are averaged over $\mathrm{p}_{\mathrm{IOI}}$. Over 100,000 dataset examples, GPT-2 small has mean logit difference of 3.56 (IO predicted over S 99.3% of the time), and mean IO probability of 49%.

Transformer architecture. GPT-2 small is a decoder-only transformer with 12 layers and 12 attention heads per attention layer. In this work, we mostly focus on understanding the behavior of attention heads, which we describe below using notation similar to Elhage et al. (2021). We leave a full description of the model to Appendix G.

The input x_0 to the transformer is a sum of position and token embeddings and lies in $\mathbb{R}^{N\times d}$, where N is the number of tokens in the input and d is the model dimension. This input embedding is the initial value of the *residual stream*, which all attention layers and MLPs read from and write to. Attention layer i of the network takes as input $x_i \in \mathbb{R}^{N\times d}$, the value of the residual stream at layer i. The attention layer output can be decomposed into the sum of attention heads $h_{i,j}$. If the output of the attention layer is $y_i = \sum_j h_{i,j}(x_i)$, then the residual stream is updated to $x_i + y_i$.

Focusing on individual heads, each head $h_{i,j}$ is parametrized by four matrices $W_Q^{i,j}, W_K^{i,j}, W_O^{i,j} \in \mathbb{R}^{d\times\frac{d}{H}}$ and $W_V^{i,j} \in \mathbb{R}^{\frac{d}{H}\times d}$. We rewrite these parameters as low-rank matrices in $\mathbb{R}^{d\times d}$: $W_{OV}^{i,j} = W_O^{i,j}W_V^{i,j}$ and $W_{QK}^{i,j} = W_Q^{i,j}(W_K^{i,j})^T$. The QK matrix is used to compute the attention pattern $A_{i,j} \in \mathbb{R}^{N\times N}$ of head (i,j), while the OV matrix determines what is written into the residual stream. At the end of the forward pass, a layer norm is applied before the unembed matrix W_U projects the residual stream into logits.

2.1 Circuits and Knockouts

In mechanistic interpretability, we want to understand the correspondence between the components of a model and human-understandable concepts. A useful abstraction for this goal is *circuits*. If we think of a model as a computational graph M where nodes are terms in its forward pass (neurons, attention heads, embeddings, etc.) and edges are the interactions between those terms (residual connections, attention, projections, etc.), a circuit C is a subgraph of M responsible for some behavior (such as completing the IOI task). This definition of a circuit is more coarse-grained than the one presented in Olah et al. (2020), where nodes are features (meaningful directions in the latent space of a model) instead of model components.

Just as the entire model M defines a function $M(x)$ from inputs to logits, we also associate each circuit C with a function $C(x)$ via *knockouts*. A knockout removes a set of nodes K in a computational graph M with the goal of "turning off" nodes in K but capturing all other computations in M. Thus, $C(x)$ is defined by knocking out all nodes in $M\backslash C$ and taking the resulting logit outputs in the modified computational graph.

A first naïve knockout approach consists of simply deleting each node in K from M. The net effect of this removal is to *zero ablate* K, meaning that we set its output to 0. This naïve approach has an important limitation: 0 is an arbitrary value, and subsequent nodes might rely on the average activation value as an implicit bias term. Perhaps because of this, we find zero ablation to lead to noisy results in practice.

To address this, we instead knockout nodes through *mean ablation*: replacing them with their average activation value across some reference distribution, similar to the bias correction method used